

Intermediate (real branching) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `conditional_routing_practice.py` : a plain edge always goes to the same next node — the moment the graph needs to decide, a conditional edge is the one piece of machinery every real agent graph, including Doc07's ReAct loop rebuilt as a graph, actually needs. **If not:** `agent_loop_to_graph` would be the first place you ever wired a decision into a graph, with no smaller example to check it against.

Read both depths — they're not "wrong, right," they're 2 real, valid ways to solve the same problem, with real tradeoffs between them.

Basic Version

Approach 1 — the direct way

```
# conditional_routing_practice.py – Intermediate section
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class GraphState(TypedDict):
    flag: bool
    message: str

def node_decide(state):
    return {}

def node_a(state):
    return {"message": "went down path A"}

def node_b(state):
    return {"message": "went down path B"}

def route(state):
```

```

    if state["flag"]:
        return "path_a"
    return "path_b"

builder = StateGraph(GraphState)
builder.add_node("node_decide", node_decide)
builder.add_node("node_a", node_a)
builder.add_node("node_b", node_b)
builder.add_edge(START, "node_decide")
builder.add_conditional_edges(
    "node_decide", route, {"path_a": "node_a", "path_b": "node_b"},
)
builder.add_edge("node_a", END)
builder.add_edge("node_b", END)

graph = builder.compile()
print(graph.invoke({"flag": True, "message": ""}))
print(graph.invoke({"flag": False, "message": ""}))

```

Expected output:

```

{'flag': True, 'message': 'went down path A'}
{'flag': False, 'message': 'went down path B'}

```

`node_decide` doesn't change state at all — its only purpose here is to be the node the conditional edge attaches to. `route` never gets registered as a node; it's just the function `add_conditional_edges` calls to pick the next node's name.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — type hints, and a routing function name that reads clearly

```
# conditional_routing_practice.py – Intermediate section
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class GraphState(TypedDict):
    flag: bool
    message: str

def node_decide(state: GraphState) -> dict:
    return {}

def node_a(state: GraphState) -> dict:
    return {"message": "went down path A"}

def node_b(state: GraphState) -> dict:
    return {"message": "went down path B"}

def route_on_flag(state: GraphState) -> str:
    if state["flag"]:
        return "path_a"
    return "path_b"

builder = StateGraph(GraphState)
builder.add_node("node_decide", node_decide)
builder.add_node("node_a", node_a)
builder.add_node("node_b", node_b)
builder.add_edge(START, "node_decide")
builder.add_conditional_edges(
    "node_decide", route_on_flag, {"path_a": "node_a", "path_b": "node_b"},
)
builder.add_edge("node_a", END)
```

```
builder.add_edge("node_b", END)

graph = builder.compile()

for flag_value in (True, False):
    result = graph.invoke({"flag": flag_value, "message": ""})
    print(f"flag={flag_value} -> {result['message']}")
```

Expected output:

```
flag=True -> went down path A
flag=False -> went down path B
```

Difference from Basic: same graph and same result, with full type hints on every function and a routing function named `route_on_flag` instead of the generic `route` — worth doing once a file has more than one conditional edge, so it's clear at a glance which state field each one branches on. The two `invoke` calls are also collapsed into a small loop over both flag values instead of being written out twice, since they're doing the same check with different input.

Approach 2 — Literal -typed routing

Story: `route_on_flag`'s return type is just `str`, so nothing stops it from returning any string, including one the mapping was never built to handle — a mismatch a type checker can catch before you ever run the code. **If not:** a typo or a dropped mapping entry would surface as a runtime `KeyError` the first time that branch actually got taken, not while you were writing the code.

```
# conditional_routing_practice.py – Intermediate section
from typing import TypedDict, Literal
from langgraph.graph import StateGraph, START, END

class GraphState(TypedDict):
    flag: bool
    message: str

def node_decide(state: GraphState) -> dict:
    return {}

def node_a(state: GraphState) -> dict:
    return {"message": "went down path A"}

def node_b(state: GraphState) -> dict:
```

```

    return {"message": "went down path B"}

def route_on_flag(state: GraphState) -> Literal["path_a", "path_b"]:
    if state["flag"]:
        return "path_a"
    return "path_b"

builder = StateGraph(GraphState)
builder.add_node("node_decide", node_decide)
builder.add_node("node_a", node_a)
builder.add_node("node_b", node_b)
builder.add_edge(START, "node_decide")
builder.add_conditional_edges(
    "node_decide", route_on_flag, {"path_a": "node_a", "path_b": "node_b"},
)
builder.add_edge("node_a", END)
builder.add_edge("node_b", END)

graph = builder.compile()

for flag_value in (True, False):
    result = graph.invoke({"flag": flag_value, "message": ""})
    print(f"flag={flag_value} -> {result['message']}")

```

Expected output:

```

flag=True -> went down path A
flag=False -> went down path B

```

Same runtime behavior as Approach 1. The difference only shows up if you break something: change the mapping to `{"path_a": "node_a"}` (dropping `"path_b"`) while `route_on_flag` can still return `"path_b"` — a type checker now flags that mismatch immediately, because `route_on_flag` 's declared return type includes a value the mapping doesn't handle.

Approach 3 — routing on more than one field

Story: most real routing decisions aren't a single boolean — this is the realistic shape, several conditions collapsed into one decision, still expressed as plain, readable Python inside one function.

If not: the first time a real routing function needed 2 fields instead of 1, you'd be improvising the pattern from scratch instead of reusing one you already practiced.

```

# conditional_routing_practice.py – Intermediate section
from typing import TypedDict, Literal

```

```

from langgraph.graph import StateGraph, START, END

class GraphState(TypedDict):
    flag: bool
    urgent: bool
    message: str

def node_decide(state: GraphState) -> dict:
    return {}

def node_a(state: GraphState) -> dict:
    return {"message": "path A (urgent)"}

def node_b(state: GraphState) -> dict:
    return {"message": "path A (normal)"}

def node_c(state: GraphState) -> dict:
    return {"message": "path B"}

def route_on_flag(
    state: GraphState,
) -> Literal["path_a_urgent", "path_a_normal", "path_b"]:
    if not state["flag"]:
        return "path_b"
    if state["urgent"]:
        return "path_a_urgent"
    return "path_a_normal"

builder = StateGraph(GraphState)
builder.add_node("node_decide", node_decide)
builder.add_node("node_a", node_a)
builder.add_node("node_b", node_b)
builder.add_node("node_c", node_c)
builder.add_edge(START, "node_decide")
builder.add_conditional_edges(
    "node_decide",
    route_on_flag,
    {"path_a_urgent": "node_a", "path_a_normal": "node_b", "path_b": "node_c"},
)
builder.add_edge("node_a", END)

```

```

builder.add_edge("node_b", END)
builder.add_edge("node_c", END)

graph = builder.compile()
print(graph.invoke({"flag": True, "urgent": True, "message": ""})["message"])
print(graph.invoke({"flag": True, "urgent": False, "message": ""})["message"])
print(graph.invoke({"flag": False, "urgent": False, "message": ""})["message"])

```

Expected output:

```

path A (urgent)
path A (normal)
path B

```

The routing function still returns one string, and the mapping still has one key per real destination — it's just that the `if / else` deciding which string to return now checks two fields instead of one. This is the realistic shape of most routing functions in a real graph: several conditions collapsed into one decision, still expressed as plain, readable Python inside one function.

Difference from Approach 1: Approach 1's `route_on_flag` returns a plain `str`, so nothing connects its possible outputs to the mapping's actual keys except careful reading. Approach 2 adds a `Literal` return type so that connection is checked by tooling, not just by eye. Approach 3 shows the same pattern scaled up to a routing decision based on two fields instead of one — same shape, one more `if` branch, and one more entry in both the `Literal` and the mapping.

Which one should you actually write? Basic's plain `-> str` is fine for a quick script or a one-off exercise. The moment a conditional edge's mapping lives more than a few lines away from its routing function — which is normal in a real file — add the `Literal` return type from Approach 2; it costs one import and catches a whole category of "routing function returns a string nothing handles" bugs before you ever run the code. Approach 3's multi-condition routing is the shape you'll actually write in Project 3 and beyond — most real routing decisions aren't a single boolean.